



STDISCM P3

Group 27 Bautista, Bilbao, Lacson, & Sandoval





Key Implementation Steps

- **Architecture:** Cross-platform Distributed System.
 - Producer: macOS (Client).
 - Consumer: Ubuntu Linux VM (Server).
- **Protocol Design:** Defined video.proto using Protocol Buffers for structured binary streaming.
- **Concurrency Model:**
 - Producer: Spawns P concurrent goroutines to read from independent folders.
 - Consumer: Spawns C worker goroutines to process the job queue.
 - Web GUI: Browser-based interface (HTML/JS) polling a JSON API for real-time updates without page reloads.
- **Bonus Integration:**
 - FFmpeg: Integrated via os/exec for video compression.
 - SHA-256: Implemented content hashing for duplicate detection.





Queueing Details (Leaky Bucket)



Design Pattern

- Fixed capacity buffer.
- Input arrives faster than processing → Overflow (Drop).



Implementation

Buffered Channel in Go (make(chan *UploadJob, size)).



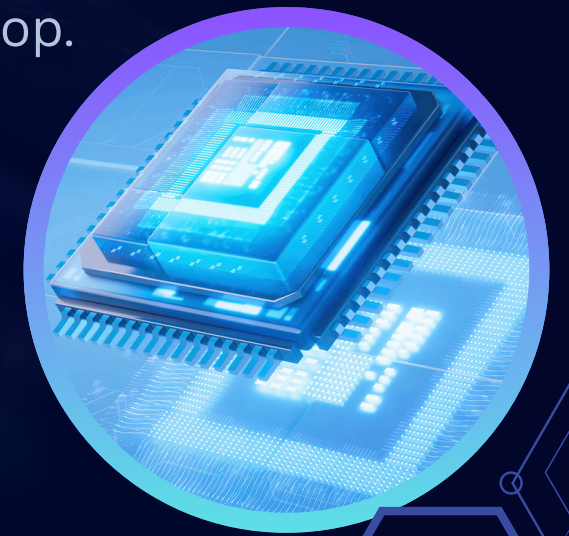
Logic Flow

- gRPC Handler receives a request.
- Non-blocking Select: Attempts to push job to channel.
- Success: Job enters queue; Worker picks it up.
- Failure (Default Case): Channel is full; Request is rejected immediately.



Backpressure (Bonus)

Returns explicit "Queue Full" message to Producer, triggering a retry loop.





Producer & Consumer Concepts

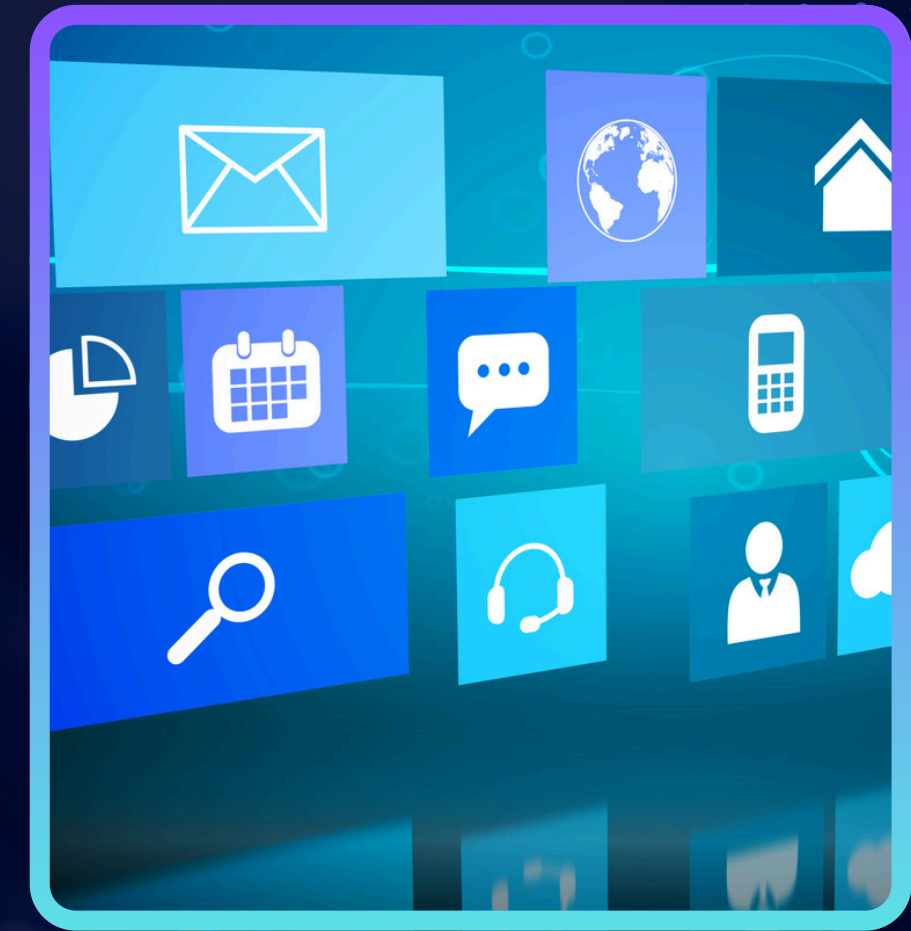


► The Producer

Role: Reads video files, calculates SHA-256 hashes, and streams data chunks.

Decoupling: Does not know how the video is stored or processed; only knows the gRPC interface.

Concurrency: Multiple threads act as independent data sources.



► Consumer

Role: Receives streams, writes to disk, and runs CPU-intensive tasks (Compression).

Worker Pool: Decouples request reception (gRPC) from request processing (FFmpeg).

State Management: Maintains the source of truth for the GUI (list of uploaded videos).



Synchronization Mechanisms



Go Channels (chan):

Primary synchronization tool. Safely passes data (UploadJob) between the gRPC Server (Producer side of the queue) and Workers (Consumer side of the queue) without locks.



Mutex (sync.RWMutex):

Protects Shared State: uploadedVideos (Slice) and uploadedHashes (Map). Why: Multiple workers write results + GUI reads results simultaneously. Prevents race conditions.



WaitGroups (sync.WaitGroup):

Used in Producer to ensure the main program waits for all upload threads to complete before exiting.



► Why gRPC?

Streaming Support: stream VideoChunk allows uploading large files (4K video) without loading the entire file into memory.

Strong Typing: Protocol Buffers (.proto) ensure strict contract between macOS and Linux (e.g., handling binary data, integers, strings).

Performance: Uses HTTP/2 and binary serialization (Protobuf), which is lighter and faster than JSON/REST for file transfers.

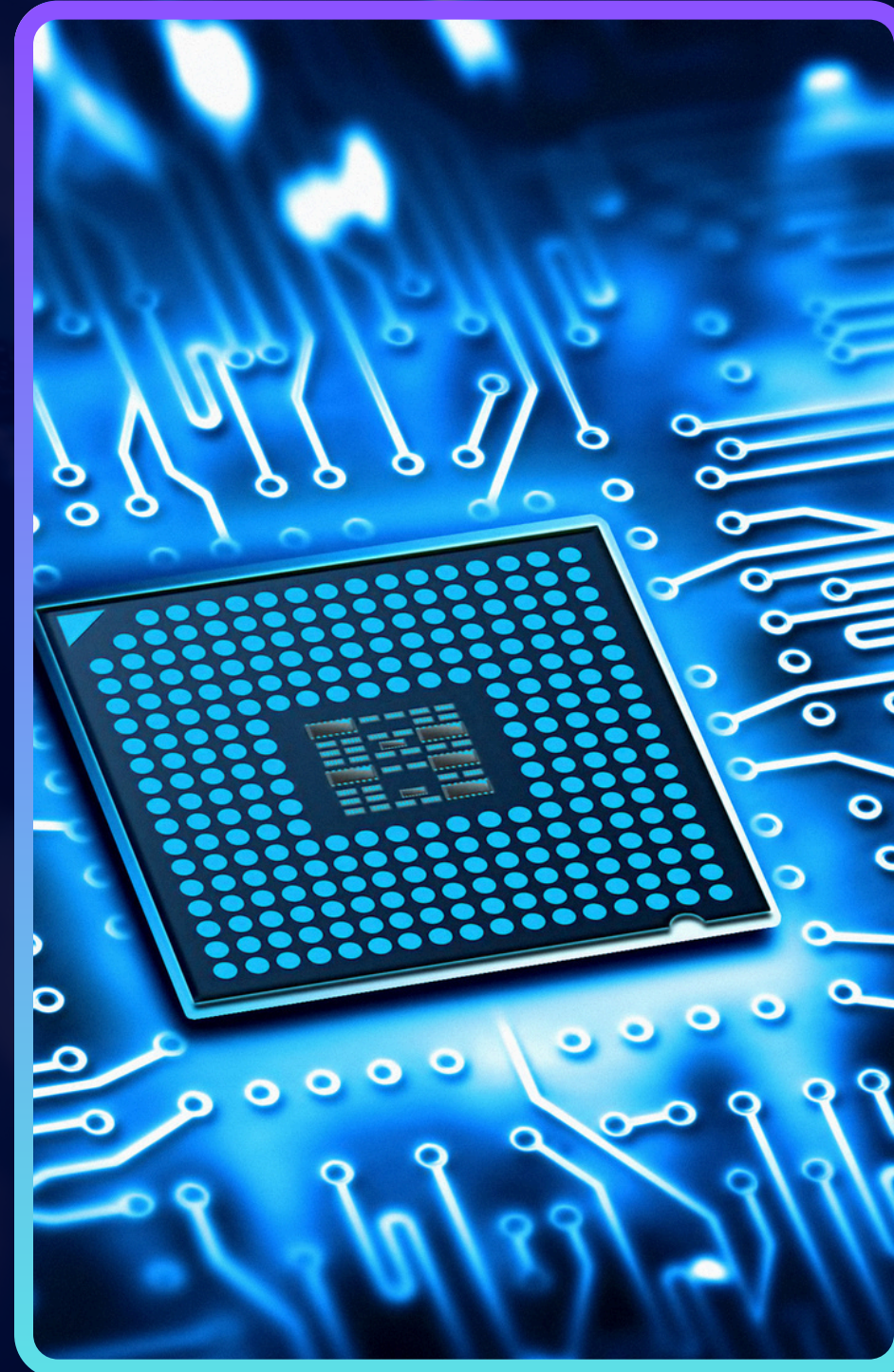


Implementation & Justification of gRPC

► Implementation

Service: rpc UploadVideo (stream VideoChunk) returns (UploadStatus)

Chunking: File is split into 64KB buffers for efficient network transmission.



Fin.